

CLRS Chapter 6 — Heapsort Study Guide

Study Guide: CLRS Introduction to Algorithms (4th Ed.) — Chapter 6: Heapsort

Generated 2026-06-09. Subject: Computer Science (Algorithms).
Chapter 6 — Heapsort.

Chapter Overview

Heapsort combines the best of merge sort ($O(n \lg n)$ worst-case time) and insertion sort (in-place). It introduces the **heap** data structure, which also serves as an efficient priority queue.

Ch. 6 — Heapsort

Named Entities

- **Heap:** A nearly complete binary tree viewed as an array; used for heapsort and priority queues.
- **Max-heap:** A heap where $A[\text{PARENT}(i)] \geq A[i]$ for every node $i \neq \text{root}$ (largest element at root).
- **Min-heap:** A heap where $A[\text{PARENT}(i)] \leq A[i]$ for every node $i \neq \text{root}$ (smallest element at root).
- **Max-heap property:** The value of a node is at most the value of its parent.
- **Min-heap property:** The value of a node is at least the value of its parent.
- **Height of a node in a heap:** Number of edges on the longest simple downward path to a leaf.
- **Height of a heap:** Height of its root ($= \lfloor \lg n \rfloor$ for an n -element heap).
- **Priority queue:** Data structure maintaining a set S of elements, each with an associated key.
- **Handle:** Additional information mapping application objects to/from array indices in the heap.
- **Heap size:** Attribute $A.\text{heap-size}$ indicating how many elements of the array are valid heap elements.

Processes / Algorithms / Pathways

PARENT, LEFT, RIGHT

- **Type:** Helpers
- **Goal:** Compute parent, left-child, right-child indices from node index i
- **Steps:**
 - PARENT(i): return $\lfloor i/2 \rfloor$
 - LEFT(i): return $2i$
 - RIGHT(i): return $2i + 1$
- **Complexity:** $O(1)$

MAX-HEAPIFY(A, i)

- **Type:** Core subroutine
- **Goal:** Maintain max-heap property — assumes LEFT(i) & RIGHT(i) are max-heaps but $A[i]$ may be smaller than children

- **Steps:**

```
MAX-HEAPIFY( $A, i$ )
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

- **Complexity:** Time $O(\lg n) = O(h)$ on node of height h . Recurrence: $T(n) \leq T(2n/3) + \Theta(1) \rightarrow O(\lg n)$ by Master Theorem Case 2.
- **Worst-case:** $\Omega(\lg n)$ — can force a path from root to leaf (Exercise 6.2-7).

BUILD-MAX-HEAP(A, n)

- **Type:** Algorithm
- **Goal:** Convert unordered array $A[1:n]$ into max-heap in linear time
- **Steps:**

```
BUILD-MAX-HEAP( $A, n$ )
1   $A.\text{heap-size} = n$ 
2  for  $i = \lfloor n/2 \rfloor$  down to 1
3      MAX-HEAPIFY( $A, i$ )
```

- **Loop invariant:** At start of each iteration, nodes $i+1, i+2, \dots, n$ are roots of max-heaps.
- **Complexity:** Time $O(n)$. Tighter analysis: at most $\lceil n/2^{(h+1)} \rceil$ nodes of height h , each costing $O(h) \rightarrow \sum h/2^h$ converges.
- **Key insight:** Leaves (indices $\lfloor n/2 \rfloor + 1$ through n) are trivially max-heaps.

HEAPSORT(A, n)

- **Type:** Algorithm (sorting, in-place)
- **Goal:** Sort array $A[1:n]$ using a max-heap
- **Steps:**

```

HEAPSORT(A, n)
1  BUILD-MAX-HEAP(A, n)
2  for i = n down to 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size - 1
5      MAX-HEAPIFY(A, 1)

```

- **Loop invariant:** $A[1:i]$ is a max-heap containing the i smallest elements; $A[i+1:n]$ contains the $n-i$ largest elements sorted.
- **Complexity:** Time $O(n \lg n)$ [BUILD-MAX-HEAP $O(n)$ + $n-1 \times$ MAX-HEAPIFY $O(\lg n)$], Space $O(1)$ in-place
- **Best-case:** $\Omega(n \lg n)$ for distinct elements (Exercise 6.4-5).

MAX-HEAP-MAXIMUM(A)

- **Goal:** Return largest element (root) in $\Theta(1)$
- **Steps:** Return $A[1]$ (check heap-size ≥ 1)

MAX-HEAP-EXTRACT-MAX(A)

- **Goal:** Remove and return the largest element
- **Steps:**

```

MAX-HEAP-EXTRACT-MAX(A)
1  max = MAX-HEAP-MAXIMUM(A)
2  A[1] = A[A.heap-size]
3  A.heap-size = A.heap-size - 1
4  MAX-HEAPIFY(A, 1)
5  return max

```

- **Complexity:** $O(\lg n)$

MAX-HEAP-INCREASE-KEY(A, x, k)

- **Goal:** Increase key of object x to k (assumes $k \geq$ current key)
- **Steps:**

```
MAX-HEAP-INCREASE-KEY(A, x, k)
1  if k < x.key  error "new key smaller"
2  x.key = k
3  find index i of x in array A
4  while i > 1 and A[PARENT(i)].key < A[i].key
5      exchange A[i] with A[PARENT(i)]
6      i = PARENT(i)
```

- **Complexity:** $O(\lg n)$ — path from node to root has length $O(\lg n)$

MAX-HEAP-INSERT(A, x, n)

- **Goal:** Insert new object x into max-heap
- **Steps:**

```
MAX-HEAP-INSERT(A, x, n)
1  if A.heap-size == n  error "heap overflow"
2  A.heap-size = A.heap-size + 1
3  k = x.key
4  x.key =  $-\infty$ 
5  A[A.heap-size] = x
6  map x to index heap-size
7  MAX-HEAP-INCREASE-KEY(A, x, k)
```

- **Complexity:** $O(\lg n)$

Classifications & Hierarchies

- **Heap types:** Max-heap (root = largest) vs Min-heap (root = smallest)
- **Priority queue types:** Max-priority queue vs Min-priority queue
- **Max-priority queue operations:** INSERT, MAXIMUM, EXTRACT-MAX, INCREASE-KEY
- **Min-priority queue operations:** INSERT, MINIMUM, EXTRACT-MIN, DECREASE-KEY

Comparisons & Trade-offs

Dimension	Heapsort	Merge sort	Insertion sort
Worst-case time	$O(n \lg n)$	$O(n \lg n)$	$O(n^2)$
In-place	Yes	No	Yes
Stable	No	Yes	Yes
Best for	Worst-case guarantee	Stable sort needed	Nearly sorted input

Dimension	Heapsort	Merge sort	Insertion sort
Practical speed	Slower than quicksort (poor cache locality)	—	Fast for small n
Dimension	Max-priority queue (heap)	Unsorted array	Sorted array
INSERT	$O(\lg n)$	$O(1)$	$O(n)$
EXTRACT-MAX	$O(\lg n)$	$O(n)$	$O(1)$
MAXIMUM	$\Theta(1)$	$O(n)$	$\Theta(1)$

Formulas & Equations

Height of an n-element heap

$$\lfloor \lg n \rfloor$$

Max nodes of height h in any n-element heap

$$\lfloor n / 2^{(h+1)} \rfloor$$

Running time of BUILD-MAX-HEAP (tight)

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \lfloor n/2^{(h+1)} \rfloor \cdot O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} h/2^h\right) = O(n)$$

Heap recurrence for MAX-HEAPIFY

$$T(n) \leq T(2n/3) + \Theta(1) \rightarrow T(n) = O(\lg n)$$

Rules, Laws & Theorems

- **Lemma (complete binary tree):** An n-element heap has height $\lfloor \lg n \rfloor$.
- **Lemma (leaf positions):** Leaves are nodes indexed by $\lfloor n/2 \rfloor + 1$ through n.
- **Master Theorem (MAX-HEAPIFY):** $T(n) = T(2n/3) + \Theta(1) \rightarrow$ Master Theorem Case 2 ($a=1$, $b=3/2$, $f(n)=\Theta(1)$) $\rightarrow \Theta(\lg n)$.
- **Corollary 8.2:** Heapsort and merge sort are asymptotically optimal comparison sorts (lower bound $\Omega(n \lg n)$).

Data Structures & Types

Max-heap (array representation)

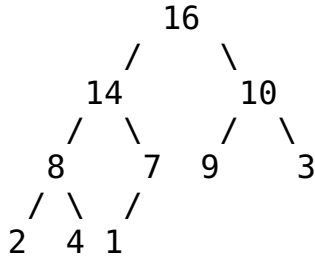
- **Structure:** Nearly complete binary tree stored in array $A[1:n]$; $A[1]$ is root.
- **Index arithmetic:** $\text{PARENT}(i) = \lfloor i/2 \rfloor$, $\text{LEFT}(i) = 2i$, $\text{RIGHT}(i) = 2i+1$.
- **Operations:** MAX-HEAPIFY $O(\lg n)$, BUILD-MAX-HEAP $O(n)$, HEAPSORT $O(n \lg n)$, INSERT $O(\lg n)$, EXTRACT-MAX $O(\lg n)$, INCREASE-KEY $O(\lg n)$, MAXIMUM $\Theta(1)$

Visual Patterns

Array view of a max-heap ($A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$):

Index: 1 2 3 4 5 6 7 8 9 10
Value: 16 14 10 8 7 9 3 2 4 1

Tree view:



Heapsort on $[5, 13, 2, 25, 7, 17, 20, 8, 4]$ ($n=9$):

BUILD-MAX-HEAP $\rightarrow [25, 13, 20, 8, 7, 17, 2, 5, 4]$

$i=9$: swap $25 \leftrightarrow 4 \rightarrow [4, 13, 20, 8, 7, 17, 2, 5 \mid 25]$; MAX-HEAPIFY $\rightarrow$
 $[20, 13, 17, 8, 7, 4, 2, 5 \mid 25]$
 $i=8$: swap $20 \leftrightarrow 5 \rightarrow [5, 13, 17, 8, 7, 4, 2 \mid 20, 25]$; MAX-HEAPIFY $\rightarrow$
 $[17, 13, 5, 8, 7, 4, 2 \mid 20, 25]$
 $i=7$: swap $17 \leftrightarrow 2 \rightarrow [2, 13, 5, 8, 7, 4 \mid 17, 20, 25]$; MAX-HEAPIFY $\rightarrow$
 $[13, 8, 5, 2, 7, 4 \mid 17, 20, 25]$
 $i=6$: swap $13 \leftrightarrow 4 \rightarrow [4, 8, 5, 2, 7 \mid 13, 17, 20, 25]$; MAX-HEAPIFY $\rightarrow$
 $[8, 7, 5, 2, 4 \mid 13, 17, 20, 25]$
 $i=5$: swap $8 \leftrightarrow 4 \rightarrow [4, 7, 5, 2 \mid 8, 13, 17, 20, 25]$; MAX-HEAPIFY $\rightarrow$
 $[7, 4, 5, 2 \mid 8, 13, 17, 20, 25]$
 $i=4$: swap $7 \leftrightarrow 2 \rightarrow [2, 4, 5 \mid 7, 8, 13, 17, 20, 25]$; MAX-HEAPIFY $\rightarrow$
 $[5, 4, 2 \mid 7, 8, 13, 17, 20, 25]$
 $i=3$: swap $5 \leftrightarrow 2 \rightarrow [2, 4 \mid 5, 7, 8, 13, 17, 20, 25]$; MAX-HEAPIFY $\rightarrow$
 $[4, 2 \mid 5, 7, 8, 13, 17, 20, 25]$
 $i=2$: swap $4 \leftrightarrow 2 \rightarrow [2, 4, 5, 7, 8, 13, 17, 20, 25]$ sorted

MAX-HEAPIFY($A, 2$) on heap of size 10:

Initial: $A[2]$ violates max-heap property (smaller than children)

Step 1: Find largest of $A[2]$, $A[4]$, $A[9]$

Step 2: Swap $A[2]$ with $A[4] \rightarrow$ node 4 now may violate

Step 3: Recurse on node 4, swap $A[4]$ with $A[9]$

Step 4: Node 9 is a leaf $\rightarrow$ done

Edge Cases & Common Pitfalls

- **Heap overflow/underflow:** INSERT checks array bounds; EXTRACT-MAX/MAXIMUM check heap-size ≥ 1 .
- **MAX-HEAPIFY on leaf:** When $i > A.\text{heap-size}/2$, i is a leaf $\rightarrow$ no effect, returns immediately.
- **Already sorted input:** Heapsort still takes $\Theta(n \lg n)$ time (best-case also $\Omega(n \lg n)$ for distinct elements).
- **Increasing key:** INCREASE-KEY errors if new key $<$ current key (cannot decrease key in a max-heap).

- **BUILD-MAX-HEAP' via INSERT:** Using repeated INSERT costs $\Theta(n \lg n)$, slower than BUILD-MAX-HEAP's $O(n)$.
- **Distinctness:** If elements may be equal, convert to ordered pairs to ensure deterministic behavior.
- **Heapsort vs quicksort:** Despite same asymptotic bound, quicksort is faster in practice due to better cache locality and smaller constant factors.
- **Handle maintenance:** When heap elements are moved, handles mapping objects to array indices must be updated.

End-of-Chapter Material

- **Key terms:** heap, max-heap, min-heap, heap property, height, max-priority queue, min-priority queue, handle, heapify
- **Exercises:** 6.1-1 through 6.5-11 covering heap properties, MAX-HEAPIFY correctness, BUILD-MAX-HEAP linear-time proof, HEAPSORT loop invariant, priority queue operations, d-ary heaps, Young tableaux.
- **Problems:** 6-1 (Building heap via insertion), 6-2 (d-ary heaps), 6-3 (Young tableaux).

Selected Exercise Solutions

- **6.1-1:** Min elements = 2^h , Max elements = $2^{(h+1)} - 1$ (for height h).
- **6.1-2:** Height = $\lfloor \lg n \rfloor$ because a complete binary tree of height h has between 2^h and $2^{(h+1)} - 1$ nodes.
- **6.1-4:** In a max-heap, the smallest element can be anywhere among the leaves.
- **6.1-8:** Leaves are indexed from $\lfloor n/2 \rfloor + 1$ to n .
- **6.2-2:** Recurrence for MAX-HEAPIFY on subtrees $\leq 2n/3$ nodes, giving $T(n) \leq T(2n/3) + \Theta(1)$.
- **6.4-3:** Heapsort takes $\Theta(n \lg n)$ on both already sorted (increasing) and reverse-sorted (decreasing) arrays.
- **6.5-9:** Use a min-priority queue to merge k sorted lists in $O(n \lg k)$ time.

Cross-Chapter Links

- **Requires:** Ch. 2 (Insertion sort comparison), Ch. 4 (Master Theorem for MAX-HEAPIFY recurrence), Appendix B.5.3 (nearly complete binary trees)
- **Referenced in:** Ch. 7 (quicksort comparison), Ch. 8 (lower bound, Corollary 8.2: heapsort is asymptotically optimal), Ch. 15, 21, 22 (min-priority queues via min-heaps), Ch. 16 (Fibonacci heaps), Ch. 19 (mergeable heaps)